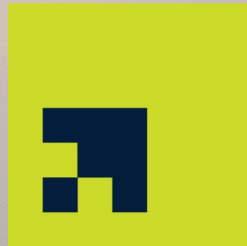


File Handling

Dr. Sanjit Kumar Saha



Gisma
University
of Applied
Sciences



File Handling

- File handling refers to the process of performing operations on a file, such as
 - creating, opening, reading, writing and closing it through a programming interface.
- It involves managing the data flow between the program and the file system on the storage device, ensuring that data is handled safely and efficiently.



Need for File Handling

- Store data permanently, even after the program ends.
- Access external files like .txt, .csv, .json, etc.
- Process large files efficiently without using much memory.
- Automate tasks like reading configs or saving outputs.



Opening a File

- To open a file, we can use ***open()*** function, which requires file-path and mode as arguments.
- Syntax:

```
f = open("filename.txt", "mode")
```

- ***filename.txt***: name (or path) of the file to be opened.
- ***mode***: mode in which you want to open the file (read, write, append, etc.).



Basic Example: Opening a File

```
f = open("geek.txt", "r")  
print(f)
```

- Code opens file `geek.txt` in read mode.
 - If the file exists, it returns a file object connected to that file;
 - If the file does not exist, Python raises a `FileNotFoundError`.



Closing a File

- ***file.close()*** method closes the file and releases the system resources.
- If the file was opened in write or append mode, closing ensures that all changes are properly saved.

```
file = open("geek.txt", "r")  
# Perform file operations  
file.close()
```



Checking File Properties

- Once the file is open, we can check some of its properties:

```
f = open("geek.txt", "r")
print("Filename:", f.name)
print("Mode:", f.mode)
print("Is Closed?", f.closed)
```

```
f.close()
print("Is Closed?", f.closed)
```

- ***f.name***: Returns the name of the file that was opened (in this case, "***geek.txt***").
- ***f.mode***: Tells us the mode in which the file was opened. Here, it's '***r***' which means read mode.
- ***f.closed***: Returns a boolean value - False when file is currently open otherwise True.



Reading a File

- Reading a file can be achieved by `file.read()` which reads the entire content of the file.
- After reading, it's good practice to close the file to free up system resources.

```
file = open("geek.txt", "r")  
content = file.read()  
print(content)  
file.close()
```




Writing a File

- Writing to a file is done using the mode **"w"**.
- This creates a new file if it doesn't exist, or overwrites the existing file if it does.
- The **write()** method is used to add content. After writing, make sure to close the file.

```
with open("geek.txt", "w") as file:  
    file.write("Hello, Python!\n")  
    file.write("File handling is easy with  
Python.")
```

```
print("File written successfully")
```

- **"w"** mode opens the file for writing (overwrites existing content if the file already exists).
- **write()** method adds new text to the file.
- When using **with**, the file closes automatically at the end of the block.



Using with Statement

- Instead of manually opening and closing the file, we can use the with statement, which automatically handles closing.
- This reduces the risk of file corruption and resource leakage.
- *Example:* Let's assume we have a file named `geek.txt` that contains text "Hello, World!".

```
with open("geek.txt", "r") as file:  
    content = file.read()  
    print(content)
```



Handling Exceptions When Closing a File

- It's important to handle exceptions to ensure that files are closed properly, even if an error occurs during file operations.
- Here, the finally block ensures the file is closed even if an error occurs.

```
try:
    file = open("geek.txt", "r")
    content = file.read()
    print(content)
except FileNotFoundError as e:
    print("Error:", e)
finally:
    file.close()
```

- **try** block contains code that may raise an error.
- **except** block handles specific errors like missing files.
- **finally** block ensures the file is always closed, even if an error occurs.



Reading CSV Files

- CSV (Comma-Separated Values) is widely used for storing tabular data. Python's csv module helps parse CSV easily. Here, instead of needing an external file, we'll simulate one using io.StringIO.
- *Example:* This code creates an in-memory CSV string, then reads it row by row.

```
import csv
import io

# Create a CSV sample in memory
csv_data = """Year,Industry,Value
2014,Manufacturing,769400
2014,Manufacturing,48000
2014,Manufacturing,12
"""

csvfile = io.StringIO(csv_data)
csvreader = csv.reader(csvfile)
for row in csvreader:
    print(row)
```



Reading JSON Files

- Reading JSON (JavaScript Object Notation) files are widely used in APIs and web applications for data transfer. Python's json module allows us to easily load JSON into dictionaries for further processing.
- *Example:* This code opens a JSON file, loads its data, and prints it as a Python dictionary.

```
import json
with open("sample1.json", "r") as jsonfile:
    data = json.load(jsonfile)
    print(data)
```




File References

- Reading from, and writing text files in Python is relatively straightforward

Mode / Option	Description
"w"	Write mode: creates file if missing, truncates (erases) if it exists
"a"	Append mode: creates file if missing, writes data always at the end
"x"	Exclusive create: creates new file, but fails with FileExistsError if it already exists
"b"	Binary flag: used with other modes (e.g., "wb", "ab") for binary files
"+"	Read/write flag: combine with other modes (e.g., "r+", "w+") for both reading and writing
encoding=	Specify text encoding (e.g., "utf-8") when working with text files
newline=	Control newline translation in text mode (e.g., "\n")